

LAPORAN PRAKTIKUM
PRAKTIKUM PENAMBANGAN DATA
PERTEMUAN 4
REGRESSION



Disusun oleh:

Nama : Hafidz Rizqullah Prasetya
NIM : 24/535493/SV/24243
Kelas : PL5A1
Dosen Pengampu : Dr. Imam Fahrurrozi, S.T., M.Cs.

PROGRAM STUDI D-IV
TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026

Daftar Isi

Daftar Isi	2
Tujuan Praktikum	3
Dasar Teori	4
2.1 Regresi	4
2.2 Algoritma Regression	4
2.2.1 Linear Regression	4
2.2.2 Decision Tree Regression	6
2.2.3 Random Forest Regression	6
2.3 Metrik Evaluasi Regression	7
2.3.1 RMSE (Root Mean Squared Error)	7
2.3.2 Coefficient of Determination (R^2)	7
2.3.3 Koefisien Korelasi Pearson (R)	8
2.3.4 Mengapa Tidak Memakai Accuracy, Precision, Recall?	8
2.4 Validation Techniques	8
Hasil dan Pembahasan	10
3.1 Percobaan Latihan Praktikum	10
3.1.1 Alat dan Bahan	10
3.1.2 Persiapan dan Impor Pustaka	10
3.1.3 Memuat Dataset KC House	11
3.1.4 Exploratory Data Analysis (EDA)	12
3.1.4.1 Deskripsi Statistik	12
3.1.4.2 Histogram Variabel Numerik	13
3.1.4.3 Korelasi Atribut Numerik	14
3.1.4.4 Pengecekan Missing Value	15
3.1.4.5 Pengecekan Atribut Kategorikal	16
3.1.5 Modelling	17
3.1.5.1 a. Linear Regression	17
3.1.5.2 b. Decision Tree Regression	22
3.1.5.3 c. Random Forest Regression	26
3.1.6 Perbandingan Performa Model KC House	31

3.1.7	Tugas dan Analisis: Car Price Prediction	32
3.1.7.1	Tujuan Penggunaan Dataset	32
3.1.7.2	Atribut Input dan Output	32
3.1.7.3	Eksplorasi Awal dan Preprocessing Dataset Car	33
3.1.7.4	Pemodelan Regresi pada Dataset Car	36
3.1.7.5	Tabel Performa Model	39
3.1.7.6	Analisis Model Terbaik	40
Kesimpulan		41
Daftar Pustaka		43

Tujuan Praktikum

Sesuai dengan Modul Pertemuan 4, tujuan pembelajaran pada praktikum ini adalah:

1. Mahasiswa mampu memahami konsep dasar regresi.
2. Mahasiswa mengetahui prinsip kerja berbagai algoritma regresi, khususnya Linear Regression, Decision Tree Regression, dan Random Forest Regression.
3. Mahasiswa dapat menerapkan berbagai algoritma regresi untuk membangun model.
4. Mahasiswa mampu mengevaluasi performa model regresi menggunakan metrik RMSE, R^2 , dan koefisien korelasi (R).

Dasar Teori

2.1 Regresi

Regresi merupakan salah satu teknik *supervised learning* yang bertujuan memodelkan hubungan antara satu atau lebih variabel independen (fitur, X) dengan variabel dependen (target, y) yang bersifat numerik atau kontinu. Model regresi belajar dari pasangan *input-output* berlabel, kemudian menghasilkan fungsi matematis yang mampu memprediksi nilai target dari data baru yang belum pernah dilihat. Contoh penerapannya meliputi prediksi harga rumah, estimasi permintaan barang, prediksi curah hujan, hingga evaluasi risiko [1, 11, 15].

Berbeda dengan klasifikasi yang keluarannya berupa label diskrit (misalnya stroke atau normal), keluaran regresi berupa nilai kontinu seperti harga, gaji, atau usia. Keduanya sama-sama termasuk *supervised learning*, sedangkan *clustering* yang bekerja tanpa label termasuk *unsupervised learning* [1, 17].

Pada regresi, dataset terdiri dari fitur input X dan target numerik y . Model dilatih pada data historis untuk meminimalkan kesalahan prediksi, lalu diuji pada data yang belum pernah dilihat untuk mengukur kemampuan generalisasinya. Karena target bersifat kontinu, evaluasi tidak bisa memakai akurasi klasifikasi, melainkan memakai metrik khusus seperti RMSE dan R^2 yang dirancang untuk menilai kualitas prediksi numerik [8, 16].

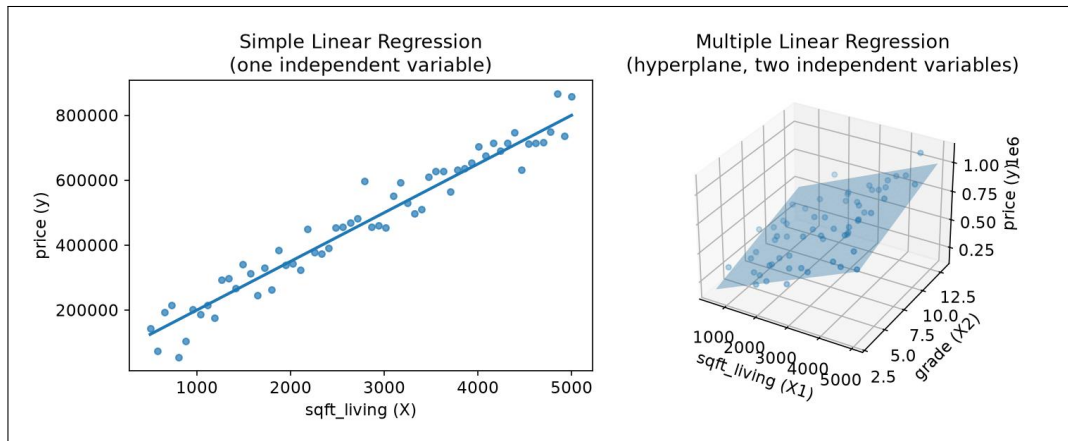
2.2 Algoritma Regression

Ada banyak metode yang dapat dipakai untuk melakukan regresi. Modul Pertemuan 4 menekankan tiga algoritma yang paling sering digunakan karena sederhana, efektif, dan terdokumentasi baik di Scikit-Learn: Linear Regression, Decision Tree Regression, dan Random Forest Regression.

2.2.1 Linear Regression

Linear Regression merupakan algoritma regresi paling dasar yang mengasumsikan hubungan antara fitur dan target bersifat linier. Model ini membangun sebuah garis (untuk satu fitur) atau *hyperplane* (untuk banyak fitur) terbaik yang meminimalkan *error* antara

prediksi dan nilai sebenarnya [5, 11].

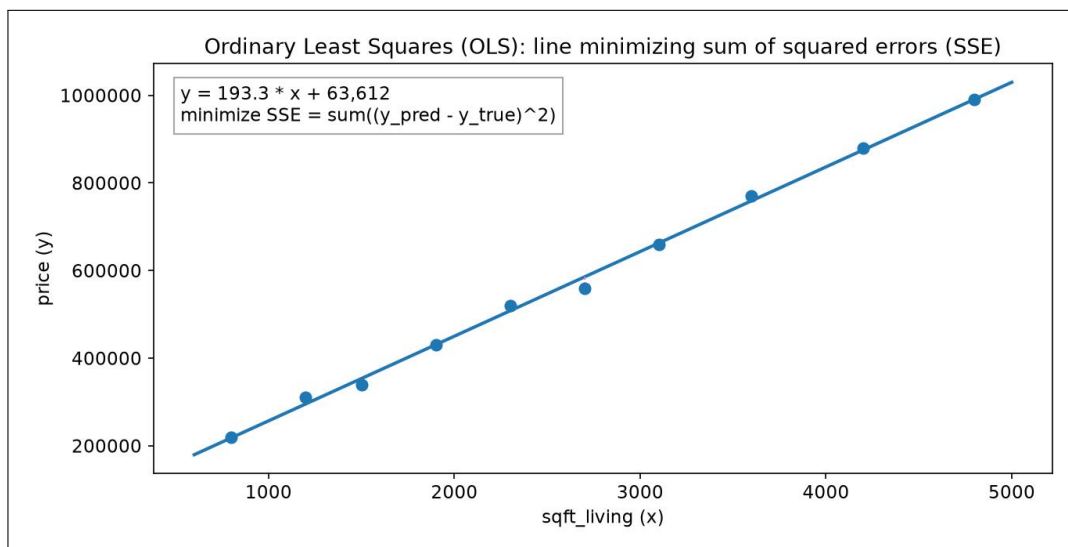


Gambar 1: Simple Linear Regression (satu variabel independen, garis lurus 2D) versus Multiple Linear Regression (dua variabel independen, bidang hyperplane 3D).

Secara matematis, model regresi linier berganda ditulis sebagai:

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n \quad (.1)$$

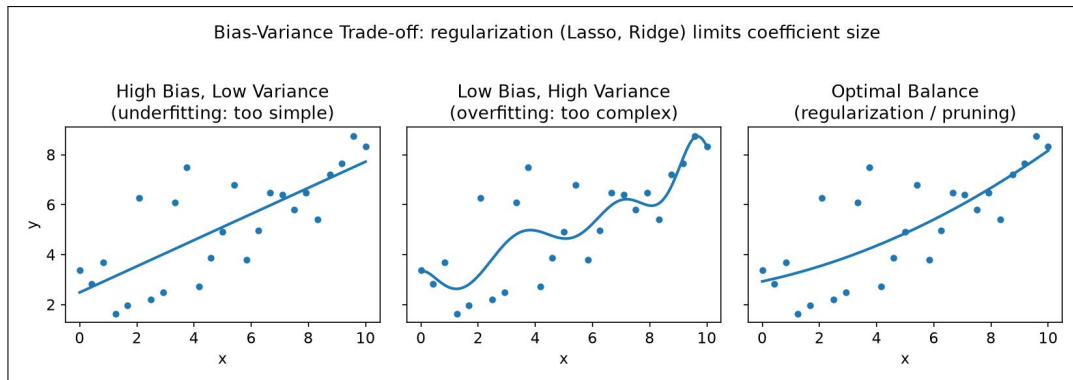
di mana b_0 adalah *intercept* (nilai dasar ketika semua fitur nol), b_n adalah koefisien regresi yang menunjukkan pengaruh fitur x_n terhadap prediksi, dan y adalah target. Model mencari koefisien terbaik menggunakan teknik *Least Squares* (kuadrat terkecil), yaitu metode yang meminimalkan jumlah kuadrat *error* [5, 11].



Gambar 2: Ordinary Least Squares (OLS): garis optimal yang memberikan nilai sum of squared errors (SSE) terendah; garis putus-putus menunjukkan residual tiap titik data.

Linear Regression cepat dan efisien untuk dataset besar, mudah dipahami dan dijelaskan (*interpretable*), serta cocok untuk hubungan yang mendekati linear. Kekurangannya, model ini kurang cocok untuk pola non-linear dan sensitif terhadap *outlier*. Karena model memberi

koefisien pada semua fitur termasuk yang daya prediksinya rendah, ia cenderung bersifat *high-variance*, *low bias*, sehingga solusinya adalah regularisasi (Lasso, Ridge, Elastic Net) yang memberi penalti pada besarnya koefisien [5, 11].



Gambar 3: Bias-variance trade-off: model terlalu sederhana mengalami underfitting (high bias), model terlalu kompleks mengalami overfitting (low bias, high variance); regularisasi membantu mencapai keseimbangan optimal.

2.2.2 Decision Tree Regression

Decision Tree Regression bekerja dengan membangun struktur pohon keputusan sebagai representasi model. Algoritma ini membagi data menjadi beberapa subset berdasarkan titik *split* yang dianggap paling optimal dalam mengurangi *error*. Setiap pembagian *node* dilakukan secara rekursif hingga mencapai *leaf node* yang berisi nilai prediksi. Pemilihan *split* biasanya dilakukan berdasarkan pengukuran *error* seperti *Mean Squared Error* (MSE), sehingga pemisahan data menghasilkan kelompok dengan variasi nilai target yang lebih homogen [6, 1, 18].

Decision Tree Regression efektif untuk memodelkan hubungan non-linear karena struktur pohon memungkinkan model membuat batas keputusan yang kompleks. Decision tree juga tidak memerlukan normalisasi atau *scaling* data dan dapat menangani fitur numerik maupun kategorikal. Kelemahan utamanya adalah kecenderungan *overfitting*, yaitu ketika model terlalu menyesuaikan diri dengan data *training* sehingga performanya menurun pada data baru. Solusinya adalah pembatasan kedalaman pohon (*max_depth*) atau *pruning* (pemangkasan pohon) [6].

2.2.3 Random Forest Regression

Random Forest Regression merupakan pengembangan dari Decision Tree yang menggunakan pendekatan *ensemble learning*, yaitu membangun banyak pohon keputusan dan menggabungkan prediksinya. Setiap pohon dibangun menggunakan sampel data yang dipilih secara acak melalui *bootstrap sampling*, serta subset fitur yang juga dipilih secara acak pada setiap pemisahan *node*. Proses ini menciptakan berbagai variasi pohon yang kemudian memberikan hasil prediksi lebih stabil dan tidak mudah *overfitting* dibanding *tree* tunggal [7, 18].

Prediksi akhir pada Random Forest diperoleh dari rata-rata hasil prediksi semua pohon (untuk kasus regresi; pada klasifikasi memakai voting mayoritas), sehingga model mampu menangkap pola kompleks tanpa mengorbankan akurasi. Random Forest juga menyediakan informasi *feature importance* yang menunjukkan fitur mana yang paling berpengaruh dalam prediksi. Karena itu algoritma ini banyak dipakai pada dataset besar dengan banyak fitur, meski memerlukan sumber daya komputasi yang lebih besar [7].

2.3 Metrik Evaluasi Regression

Dalam *machine learning* khususnya pada model regresi, diperlukan metrik evaluasi untuk menilai seberapa baik model mampu memprediksi nilai numerik dengan tepat. Evaluasi ini dilakukan dengan membandingkan hasil prediksi model terhadap nilai aktual menggunakan metrik khusus regresi [8].

2.3.1 RMSE (Root Mean Squared Error)

RMSE mengukur rata-rata jarak antara nilai prediksi dan nilai sebenarnya dengan memberikan penalti lebih besar terhadap kesalahan yang besar. RMSE merupakan akar dari *Mean Squared Error*, sehingga unitnya sama dengan unit target dan lebih mudah diinterpretasikan dalam masalah nyata. Nilai RMSE yang lebih kecil menandakan bahwa model menghasilkan prediksi yang lebih akurat [8].

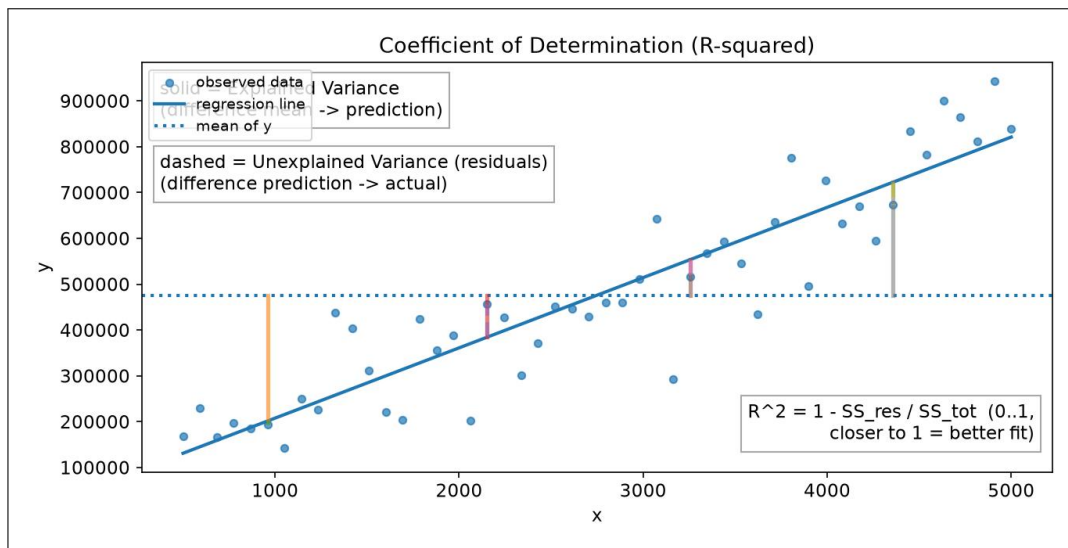
$$RMSE = \sqrt{\frac{1}{n} \sum (y_{pred} - y_{true})^2} \quad (.2)$$

Karena RMSE memberikan penalti lebih besar pada kesalahan ekstrem, metrik ini sensitif terhadap *outlier*. Sensitivitas tersebut justru menjadikan RMSE cocok digunakan pada data kontinu seperti harga rumah, di mana perbedaan nilai yang besar memberikan dampak signifikan terhadap evaluasi performa model.

2.3.2 Coefficient of Determination (R^2)

Coefficient of Determination atau R^2 menunjukkan seberapa besar proporsi variansi target yang dapat dijelaskan oleh model. Nilai R^2 berada pada rentang 0 hingga 1, di mana nilai mendekati 1 menandakan bahwa model memiliki kemampuan prediksi yang baik dan dapat menangkap pola hubungan dengan baik [8].

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} \quad (.3)$$



Gambar 4: Koefisien determinasi (R^2): proporsi explained variance (selisih rata-rata ke prediksi) terhadap total variance; sisanya adalah unexplained variance (residual).

Nilai R^2 yang semakin mendekati 1 menunjukkan bahwa model mampu menjelaskan proporsi variansi target dengan lebih baik. Sebaliknya, jika nilai R^2 rendah, berarti model kurang mampu menangkap hubungan antara fitur dan target. R^2 pada data *training* dan *testing* juga sebaiknya dibandingkan, karena selisih yang besar mengindikasikan *overfitting* [8, 1].

2.3.3 Koefisien Korelasi Pearson (R)

Koefisien korelasi Pearson (r) mengukur kekuatan dan arah hubungan linier antara dua variabel, dalam hal ini antara nilai prediksi dan nilai sebenarnya. Nilainya berada pada rentang -1 hingga 1 : semakin dekat ke 1 , semakin kuat hubungan linier positif antara prediksi dan aktual, yang berarti model semakin baik [8].

2.3.4 Mengapa Tidak Memakai Accuracy, Precision, Recall?

Metrik seperti *accuracy*, *precision*, dan *recall* dirancang untuk target diskrit berbasis *confusion matrix* (TP, TN, FP, FN). Pada regresi, prediksi berupa bilangan kontinu sehingga peluang prediksi sama persis dengan nilai aktual praktis nol dan konsep benar/salah tidak terdefinisi. Oleh karena itu, performa model regresi harus diukur dari besarnya *error* (RMSE) dan proporsi variansi yang dijelaskan model (R^2), bukan dari proporsi prediksi yang tepat [8].

2.4 Validation Techniques

Performa model dapat dipengaruhi oleh ukuran data latih dan uji. Teknik validasi umum yang dipakai pada praktikum ini adalah *holdout*: data dibagi menjadi training set (70%) dan testing set (30%). Model dilatih pada training set dan dievaluasi pada testing set yang tidak

pernah dilihat selama pelatihan. Parameter `random_state=42` dipakai agar pembagian data tetap konsisten setiap kali kode dijalankan [1, 16, 19].

Hasil dan Pembahasan

3.1 Percobaan Latihan Praktikum

Pada bagian ini saya mendokumentasikan percobaan regresi sesuai langkah pada Modul Pertemuan 4. Dataset yang digunakan adalah *KC House Data* dari Kaggle [12] (<https://www.kaggle.com/datasets/shivachandel/kc-house-data>) yang juga tersedia di GitHub ganjar87/data_science_practice. Dataset ini berisi data penjualan rumah di King County (termasuk Seattle, AS) dari Mei 2014 sampai Mei 2015. Setiap langkah disertai kode yang saya jalankan di Google Colaboratory beserta screenshot hasil eksekusinya. Notebook yang saya gunakan dapat diakses melalui tautan Google Colab berikut: https://colab.research.google.com/drive/17jU11pvunjthV_Ve04KyfYFfoERYGoPS?usp=sharing.

3.1.1 Alat dan Bahan

Sesuai Modul 4.3, alat dan bahan yang saya gunakan adalah:

- **Perangkat keras:** Laptop/komputer dengan akses internet.
- **Perangkat lunak:** Python 3 [2] dan Google Colaboratory sebagai lingkungan eksekusi notebook. Pustaka utama yang saya gunakan meliputi pandas [3], NumPy [4], Matplotlib, Seaborn, dan Scikit-Learn untuk pemodelan regresi [8, 9].

Dataset KC House saya akses melalui mirror GitHub agar dapat dimuat langsung dengan `pd.read_csv()` tanpa unduhan manual dari Kaggle, sedangkan dataset tugas Car Price saya akses melalui mirror SampattKumar/Linear-Regression-Car-Dataset di GitHub.

3.1.2 Persiapan dan Impor Pustaka

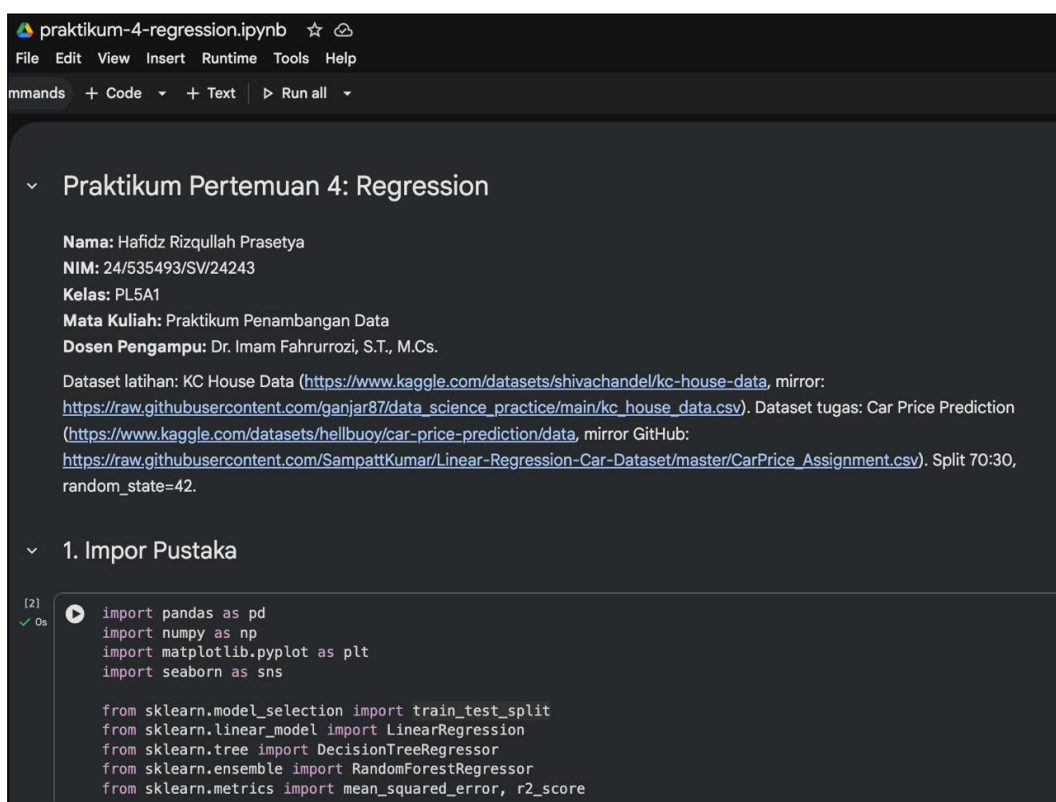
Saya menyiapkan lingkungan Python [2] dan mengimpor pustaka yang dibutuhkan untuk manipulasi data dengan pandas [3], komputasi numerik dengan NumPy [4], visualisasi, serta pemodelan regresi (Linear Regression, Decision Tree, Random Forest) beserta metrik evaluasinya.

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5
6 from sklearn.model_selection import train_test_split
7 from sklearn.linear_model import LinearRegression
8 from sklearn.tree import DecisionTreeRegressor
9 from sklearn.ensemble import RandomForestRegressor
10 from sklearn.metrics import mean_squared_error, r2_score

```

Listing .1: Impor pustaka praktikum regresi.



Gambar 1: Impor pustaka yang digunakan dalam praktikum regresi.

3.1.3 Memuat Dataset KC House

Saya memuat dataset KC House yang berisi 21.613 baris dengan 21 kolom, meliputi atribut properti (luas, kamar, grade, lokasi) dan target price (harga rumah).

```

1 df = pd.read_csv('https://raw.githubusercontent.com/ganjar87/data_science_practice/
↩ main/kc_house_data.csv')
2 print(df.shape) # (21613, 21)
3 df.head()

```

Listing .2: Memuat dataset KC House.

2. Load Dataset KC House

```
df = pd.read_csv('https://raw.githubusercontent.com/ganjar87/data_science_practice/main/kc_house_data.csv')
print(df.shape)
df.head()
```

	id	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	...	grade	sqft_above	sqft_basement	yr_built	yr_renovated	zipcode	lat	long	sqft_living15	sqft_lot15
0	7129300520	20141013T000000	221900.0	3	1.00	1180	5650	1.0	0	0	...	7	1180	0	1955	0	98178	47.5112	-122.257	1540	5650
1	6414100192	20141026T000000	638000.0	3	2.25	2570	7242	2.0	0	0	...	7	2170	400	1951	1991	98125	47.7210	-122.319	1690	7639
2	5531500480	20150225T000000	180000.0	2	1.00	770	10000	1.0	0	0	...	6	770	0	1933	0	98028	47.7379	-122.233	2720	8062
3	2487200675	20141209T000000	604000.0	4	3.00	1990	5000	1.0	0	0	...	7	1050	910	1965	0	98136	47.5208	-122.393	1360	5000
4	1954400510	20150218T000000	810000.0	3	2.00	1680	8080	1.0	0	0	...	8	1680	0	1987	0	98074	47.6168	-122.045	1800	7503

5 rows x 21 columns

Gambar 2: Lima baris pertama dataset KC House: kolom id, date, price, bedrooms, bathrooms, sqft_living, hingga grade dan sqft_above.

```
df.tail()
```

Listing .3: Lima baris terakhir dataset.

```
df.tail()
```

	id	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	...	grade	sqft_above	sqft_basement	yr_built	yr_renovated	zipcode	lat	long	sqft_living15	sqft_lot15
21808	283000018	20140521T000000	360000.0	3	2.50	1530	1131	3.0	0	0	...	8	1530	0	2009	0	98103	47.6993	-122.346	1530	1509
21809	660060120	20150223T000000	400000.0	4	2.50	2910	9813	2.0	0	0	...	8	2310	0	2014	0	98146	47.5107	-122.362	1830	7200
21810	1523300141	20140623T000000	402101.0	2	0.75	1020	1350	2.0	0	0	...	7	1020	0	2009	0	98144	47.5844	-122.299	1020	2007
21811	291310100	20150116T000000	400000.0	3	2.50	1600	2388	2.0	0	0	...	8	1600	0	2004	0	98027	47.5345	-122.069	1410	1287
21812	1523300157	20141015T000000	325000.0	2	0.75	1020	1078	2.0	0	0	...	7	1020	0	2008	0	98144	47.5841	-122.299	1020	1357

5 rows x 21 columns

Gambar 3: Lima baris terakhir dataset dengan struktur kolom yang sama, memberikan gambaran variasi data di akhir kumpulan dataset.

3.1.4 Exploratory Data Analysis (EDA)

3.1.4.1 Deskripsi Statistik

Saya melihat deskripsi statistik untuk memahami sebaran variabel numerik. Rata-rata harga rumah (price) sekitar 540.088 dengan rata-rata luas bangunan (sqft_living) sekitar 2.079. Standar deviasi price yang besar menandakan sebaran harga yang lebar, termasuk rumah-rumah yang sangat mahal.

```
display(df.describe())
```

Listing .4: Deskripsi statistik dataset.

3. Exploratory Data Analysis (EDA)

3a. Deskripsi Statistik

```
display(df.describe())
```

	id	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	condition	grade	sqft_above	sqft_basement	yr_built	yr_renovated	zipcode	lat	long	sqft_living15	sqft_lot15
count	2.161300e+04	2.161300e+04	2.161300e+04	2.161300e+04	2.161300e+04	2.161300e+04	2.161300e+04	2.161300e+04	2.161300e+04	2.161300e+04	2.161300e+04	2.161300e+04	2.161300e+04	2.161300e+04	2.161300e+04	2.161300e+04	2.161300e+04	2.161300e+04	2.161300e+04	2.161300e+04
mean	4.580302e+09	5.400811e+05	3.370842	2.114757	2079.899738	1.510897e+04	1.494309	0.007542	0.234303	3.408430	7.656873	1788.360691	291.509045	1971.005136	84.402258	98077.939805	47.580053	-122.213896		
std	2.876566e+09	3.671272e+05	0.930062	0.770163	918.440897	4.142051e+04	0.539989	0.086517	0.786318	0.692743	1.175459	828.060978	442.575043	29.373411	401.879240	53.505028	0.138564	0.140828		
min	1.000102e+06	7.500000e+04	0.000000	0.000000	290.000000	5.200000e+02	1.000000	0.000000	0.000000	1.000000	1.000000	290.000000	0.000000	1900.000000	0.000000	98001.000000	47.155900	-122.519000		
25%	2.123049e+09	3.219600e+05	3.000000	1.750000	1427.000000	5.040000e+03	1.000000	0.000000	0.000000	3.000000	7.000000	1190.000000	0.000000	1951.000000	0.000000	98033.000000	47.471000	-122.309000		
50%	3.904930e+09	4.500000e+05	3.000000	2.250000	1910.000000	7.818000e+03	1.500000	0.000000	0.000000	3.000000	7.000000	1560.000000	0.000000	1975.000000	0.000000	98065.000000	47.571800	-122.230000		
75%	7.308900e+09	6.450000e+05	4.000000	2.500000	2550.000000	1.068000e+04	2.000000	0.000000	0.000000	4.000000	8.000000	2210.000000	560.000000	1997.000000	0.000000	98118.000000	47.678000	-122.125000		
max	9.500000e+09	7.700000e+06	33.000000	8.000000	13540.000000	1.851359e+06	3.500000	1.000000	4.000000	5.000000	13.000000	9410.000000	4820.000000	2015.000000	2015.000000	98199.000000	47.777600	-121.315000		

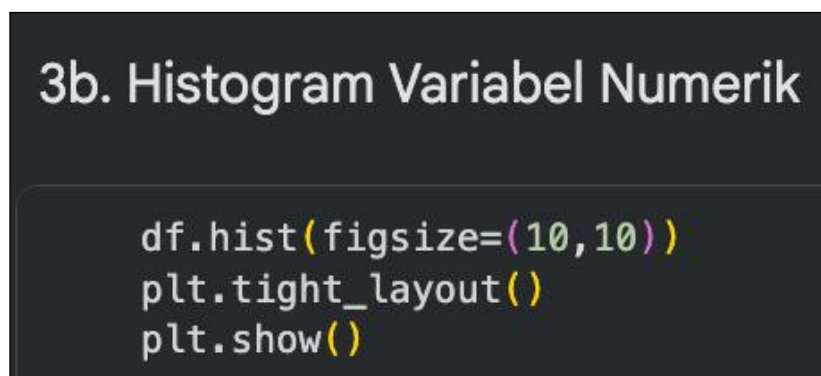
Gambar 4: Hasil deskripsi statistik: baris metrik (count, mean, std, min, 25%, 50%, 75%, max) untuk variabel numerik seperti price, bedrooms, bathrooms, dan sqft_living.

3.1.4.2 Histogram Variabel Numerik

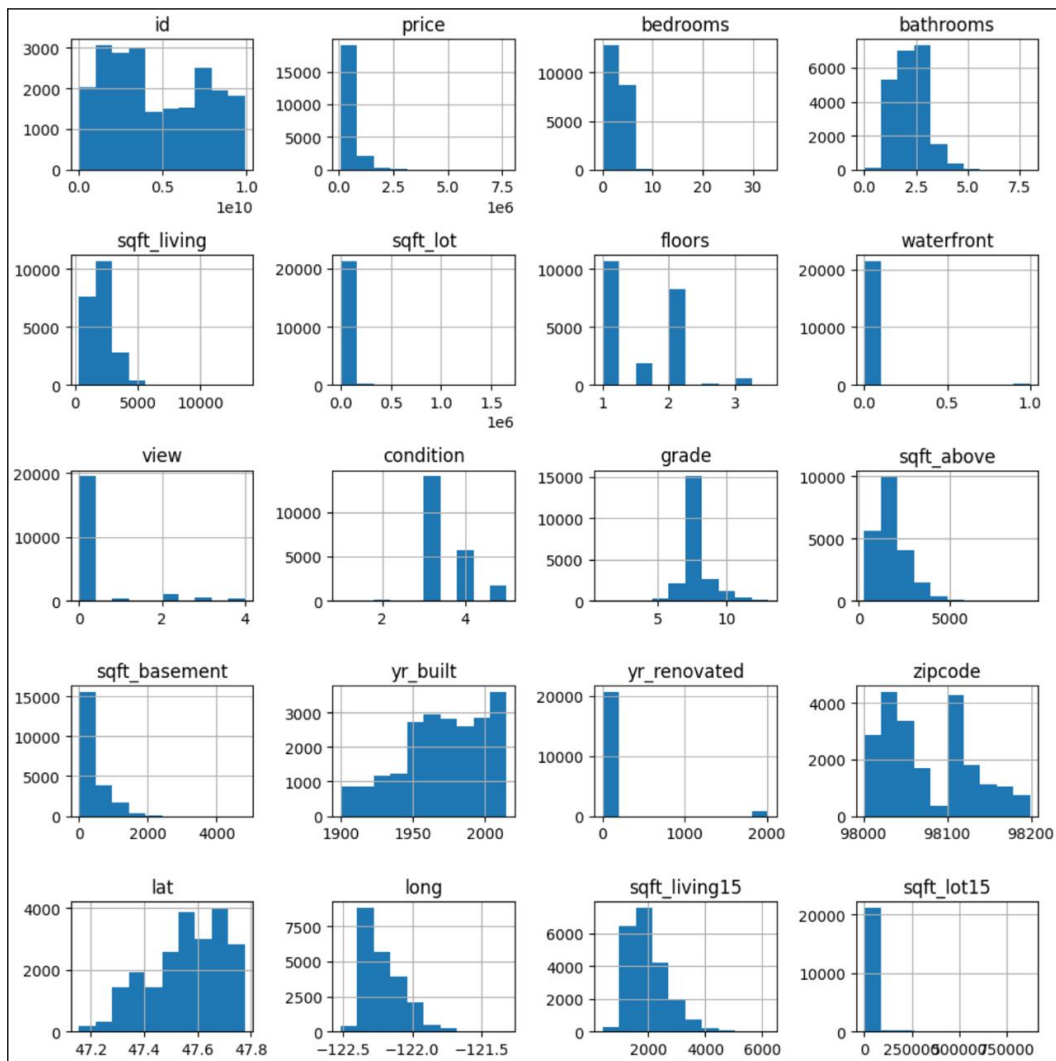
Saya memplot histogram untuk melihat distribusi tiap variabel numerik. Histogram price sangat condong ke kanan (*right-skewed*) dengan banyak outlier di nilai tinggi, sementara bedrooms dan bathrooms menunjukkan distribusi diskrit yang terkonsentrasi pada nilai-nilai kecil.

```
1 # check histogram for continuous columns
2 df.hist(figsize=(10,10))
3 plt.tight_layout()
4 plt.show()
```

Listing .5: Kode untuk menampilkan histogram.



Gambar 5: Kode untuk memplot histogram setiap kolom numerik dengan fungsi bawaan pandas/matplotlib.



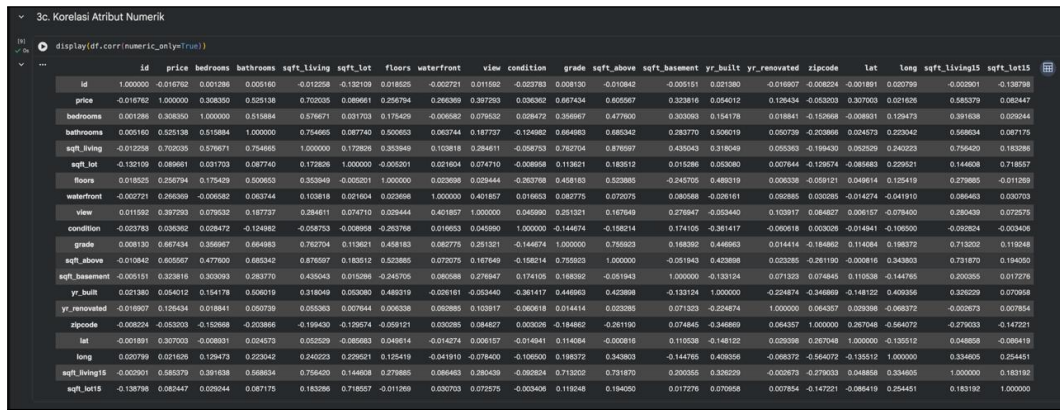
Gambar 6: Grid histogram seluruh variabel numerik: price right-skewed, variabel diskrit menumpuk di nilai kecil.

3.1.4.3 Korelasi Atribut Numerik

Saya mengecek matriks korelasi Pearson antar variabel numerik. Korelasi positif terkuat dengan price adalah sqft_living (0,70), grade (0,67), dan sqft_above (0,61): semakin luas dan semakin tinggi grade rumah, semakin tinggi harganya.

```
1 # check correlation coef
2 display(df.corr(numeric_only=True))
```

Listing .6: Korelasi atribut numerik.



Gambar 7: Matriks korelasi Pearson antar seluruh variabel numerik.

3.1.4.4 Pengecekan Missing Value

Saya mengecek nilai kosong di seluruh kolom. Hasilnya seluruh kolom memiliki 0 nilai kosong, sehingga dataset sudah bersih dan bisa langsung dipakai untuk pemodelan tanpa imputasi.

```
print(df.isnull().sum())
```

Listing .7: Pengecekan missing value.

```
3d. Missing Value

[10]
✓ Os
... id 0
    date 0
    price 0
    bedrooms 0
    bathrooms 0
    sqft_living 0
    sqft_lot 0
    floors 0
    waterfront 0
    view 0
    condition 0
    grade 0
    sqft_above 0
    sqft_basement 0
    yr_built 0
    yr_renovated 0
    zipcode 0
    lat 0
    long 0
    sqft_living15 0
    sqft_lot15 0
    dtype: int64
```

Gambar 8: Hasil pengecekan missing value: seluruh kolom 0 nilai kosong.

3.1.4.5 Pengecekan Atribut Kategorikal

Saya mengecek kolom bertipe objek/kategorikal. Setelah kolom id, date, dan price dibuang, hasilnya adalah Index kosong: tidak ada lagi kolom bertipe objek atau boolean, sehingga semua fitur sudah numerik dan tidak memerlukan encoding.

```
1 df_X = df.drop(['id', 'date', 'price'], axis=1)
2 df_y = df['price']
3 cats = df_X.select_dtypes(include=['object', 'bool']).columns
4 print(cats)
```

Listing .8: Pengecekan atribut kategorikal.

```
3e. Atribut Kategorikal

[11]
✓ Os df_X = df.drop(['id', 'date', 'price'], axis=1)
    df_y = df['price']
    cats = df_X.select_dtypes(include=['object', 'bool']).columns
    print(cats)

Index([], dtype='object')
```

Gambar 9: Hasil pengecekan atribut kategorikal: tidak ada kolom objek/boolean yang tersisa.

3.1.5 Modelling

3.1.5.1 a. Linear Regression

Proses pemodelan saya mulai dengan menentukan fitur dan target: `df_X` dibuat dengan menghapus kolom `id`, `date`, dan `price` karena `price` menjadi target prediksi sedangkan `id` dan `date` tidak informatif. Kedua variabel diubah menjadi array numerik float, lalu data dibagi 70:30 dengan `random_state=42`. Model `LinearRegression()` dilatih dengan `.fit()`, kemudian saya menghitung R^2 training dan testing, melihat koefisien dan *intercept*, serta memprediksi 10 data pertama untuk dibandingkan dengan nilai sebenarnya.

```
1 from sklearn.linear_model import LinearRegression
2 from sklearn.model_selection import train_test_split
3
4 df_X = df.drop(['id', 'date', 'price'], axis=1)
5 df_y = df['price']
6
7 X = df_X.astype(float).values
8 y = df_y.astype(float).values
9
10 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
11     ↪ random_state=42)
12
13 reg = LinearRegression()
14 reg.fit(X_train, y_train)
15
16 print('coef of determination training', reg.score(X_train, y_train))
17 print('coef of determination testing', reg.score(X_test, y_test))
18
19 print('coefficient')
20 print(reg.coef_)
21 print('intercept')
22 print(reg.intercept_)
```

```

23 print('prediction')
24 y_pred = reg.predict(X_test)
25 print(y_pred[:10])
26 print('real value')
27 print(y_test[0:10])

```

Listing .9: Kode model Linear Regression.

The screenshot shows a Jupyter Notebook cell titled "4. Modelling — Linear Regression". The code in the cell performs the following steps: imports LinearRegression and train_test_split from sklearn; drops 'id' and 'date' columns from a DataFrame; splits the data into training and testing sets (70:30 split); fits a LinearRegression model; prints the coefficient of determination for training and testing; prints the model's coefficients and intercept; and finally prints the first 10 predictions and real values. The output below the code shows that the training and testing R-squared values are nearly identical (0.6995), indicating a stable model. It also displays the coefficients for five features, the intercept, the first 10 predictions, and the first 10 real values.

```

[12]
✓ Os
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

df_X = df.drop(['id', 'date', 'price'], axis=1)
df_y = df['price']

X = df_X.astype(float).values
y = df_y.astype(float).values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

reg = LinearRegression()
reg.fit(X_train, y_train)

print('coef of determination training', reg.score(X_train, y_train))
print('coef of determination testing', reg.score(X_test, y_test))

print('coefficient')
print(reg.coef_)
print('intercept')
print(reg.intercept_)

print('prediction')
y_pred = reg.predict(X_test)
print(y_pred[:10])
print('real value')
print(y_test[0:10])

```

```

coef of determination training 0.6995155846436756
coef of determination testing 0.6994627057969904
coefficient
[-3.43081477e+04  4.03129700e+04  1.12001375e+02  9.91841247e-02
  5.27154218e+03  5.43877177e+05  5.50830616e+04  2.31460673e+04
  9.49081794e+04  7.22190668e+01  3.97823082e+01 -2.59441847e+03
  2.19209734e+01 -5.56358731e+02  5.95216324e+05 -1.96904658e+05
  1.62077488e+01 -3.30430480e-01]
intercept
6641646.708095346
prediction
[ 458597.0676416  748993.75994823 1243303.75799061 1665116.95095454
  737302.05741732 283239.58524967  831732.8758231  495383.02095346
  385779.81919014 474179.42285135]
real value
[ 365000.  865000. 1038000. 1490000.  711000.  211000.  790000.  680000.
  384500.  605000.]

```

Gambar 10: Kode pemodelan Linear Regression: pemisahan fitur/target, split 70:30, training, skor R^2 , koefisien, intercept, dan prediksi.

Hasil training menunjukkan *coef of determination training* 0,6995 dan *testing* 0,6995 yang hampir identik, artinya model stabil dan tidak overfitting. Koefisien menunjukkan pengaruh masing-masing fitur terhadap harga, sedangkan *intercept* sebesar 6.641.646,71 merupakan nilai dasar ketika semua fitur nol. Sepuluh prediksi pertama terlihat mendekati nilai sebenarnya meski ada selisih (error) pada beberapa sampel.

```

4. Modelling — Linear Regression

[12]
✓ Os
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

df_X = df.drop(['id', 'date', 'price'], axis=1)
df_y = df['price']

X = df_X.astype(float).values
y = df_y.astype(float).values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

reg = LinearRegression()
reg.fit(X_train, y_train)

print('coef of determination training', reg.score(X_train, y_train))
print('coef of determination testing', reg.score(X_test, y_test))

print('coefficient')
print(reg.coef_)
print('intercept')
print(reg.intercept_)

print('prediction')
y_pred = reg.predict(X_test)
print(y_pred[:10])
print('real value')
print(y_test[:10])

coef of determination training 0.6995155846436756
coef of determination testing 0.6994627057969904
coefficient
[-3.43081477e+04  4.03129700e+04  1.12001375e+02  9.91841247e-02
  5.27154218e+03  5.43877177e+05  5.50830616e+04  2.31460673e+04
  9.49081794e+04  7.22190668e+01  3.97823082e+01 -2.59441847e+03
  2.19209734e+01 -5.56358731e+02  5.95216324e+05 -1.96904658e+05
  1.62077488e+01 -3.30430480e-01]
intercept
6641646.708095346
prediction
[ 458597.0676416  748993.75994823 1243303.75799061 1665116.95095454
 737302.05741732 283239.58524967 831732.8758231 495383.02095346
 385779.81919014 474179.42285135]
real value
[ 365000.  865000. 1038000. 1490000.  711000.  211000.  790000.  680000.
 384500.  605000.]

```

Gambar 11: Output pelatihan Linear Regression: skor R^2 training/testing, koefisien, intercept, 10 prediksi pertama, dan 10 nilai sebenarnya.

Selanjutnya saya mengevaluasi model dengan RMSE dan R^2 . MSE dihitung dengan `mean_squared_error`, diakarkan menjadi RMSE agar satuannya sama dengan dolar, dan R^2 dihitung dengan `r2_score`.

```

1 from sklearn.metrics import mean_squared_error
2 from sklearn.metrics import r2_score
3 import numpy as np
4
5 mse = mean_squared_error(y_test, y_pred)
6 rmse = np.sqrt(mse)
7 r2 = r2_score(y_test, y_pred)
8
9 print('rmse:', rmse)
10 print('r2:', r2)

```

Listing .10: Kode evaluasi Linear Regression.

```

  Evaluasi Linear Regression (RMSE & R2)

[13]
✓ 0s
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score
import numpy as np

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print('rmse:', rmse)
print('r2:', r2)

rmse: 208296.72772118862
r2: 0.6994627057969904

```

Gambar 12: Kode evaluasi Linear Regression menggunakan RMSE dan R².

Hasilnya RMSE sebesar 208.296,73 yang berarti rata-rata kesalahan prediksi model berada di kisaran angka tersebut terhadap harga sebenarnya, dan R² sebesar 0,699 yang berarti model menjelaskan sekitar 69,9% variansi harga rumah. Masih ada sekitar 30% variansi yang belum dijelaskan model.

```

  Evaluasi Linear Regression (RMSE & R2)

[13]
✓ 0s
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score
import numpy as np

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print('rmse:', rmse)
print('r2:', r2)

rmse: 208296.72772118862
r2: 0.6994627057969904

```

Gambar 13: Hasil evaluasi Linear Regression: RMSE 208296,73 dan R² 0,699.

Terakhir saya memvisualisasikan 50 sampel pertama dalam grafik batang untuk membandingkan nilai prediksi dan nilai sebenarnya. Pola batang prediksi mengikuti fluktuasi batang nilai sebenarnya, meski ada deviasi ketinggian pada beberapa titik yang merepresentasikan error prediksi.

```

1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 data1 = pd.Series(y_test[:50].ravel())
5 data2 = pd.Series(y_pred[:50].ravel())

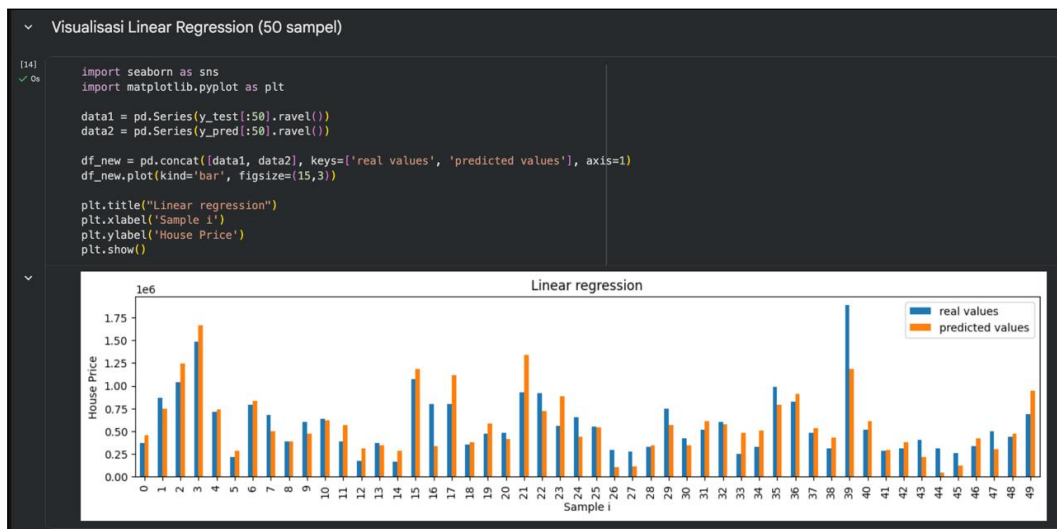
```

```

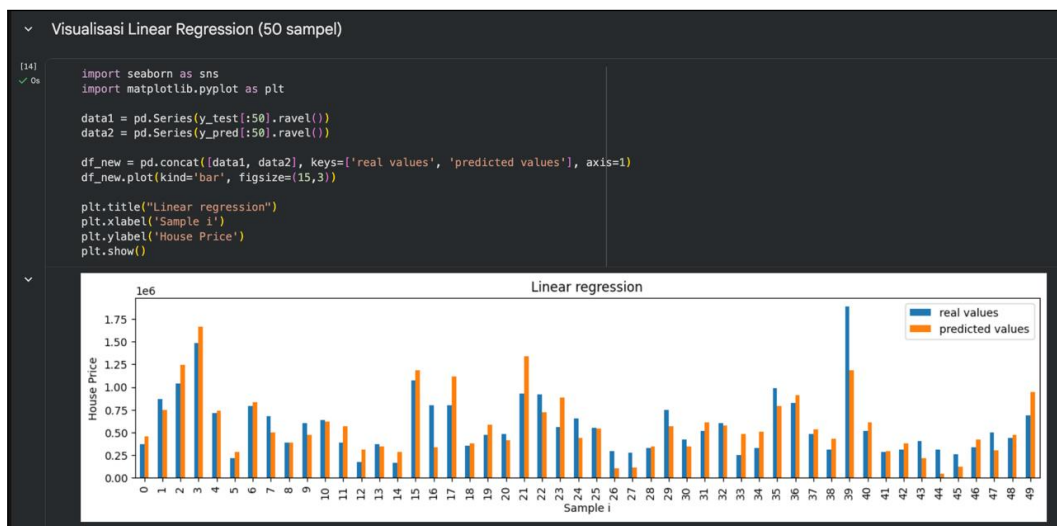
6
7 df_new = pd.concat([data1, data2], keys=['real values', 'predicted values'], axis=1)
8 # df_new.plot.bar() # bisa pakai cara 1
9 df_new.plot(kind='bar', figsize=(15,3)) # bisa pakai cara 2
10
11 plt.title("Linear regression")
12 plt.xlabel('Sample i')
13 plt.ylabel('House Price')
14 plt.show()

```

Listing .11: Kode visualisasi Linear Regression.



Gambar 14: Kode visualisasi perbandingan 50 nilai aktual dan prediksi Linear Regression.



Gambar 15: Grafik batang perbandingan real values dan predicted values untuk Linear Regression.

3.1.5.2 b. Decision Tree Regression

Untuk Decision Tree, langkah persiapan datanya identik, hanya modelnya yang diganti menjadi `DecisionTreeRegressor(max_depth=10)`. Parameter `max_depth=10` membatasi kedalaman maksimum pohon agar model tidak terlalu kompleks dan mengurangi risiko *overfitting*.

```
1 from sklearn.tree import DecisionTreeRegressor
2 from sklearn.model_selection import train_test_split
3
4 df_X = df.drop(['id', 'date', 'price'], axis=1)
5 df_y = df['price']
6
7 X = df_X.astype(float).values
8 y = df_y.astype(float).values
9
10 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
11 ↪ random_state=42)
12
13 dt = DecisionTreeRegressor(max_depth=10)
14 dt.fit(X_train, y_train)
15
16 print('coef of determination training', dt.score(X_train, y_train))
17 print('coef of determination testing', dt.score(X_test, y_test))
18
19 print('prediction')
20 y_pred = dt.predict(X_test)
21 print(y_pred[:10])
22 print('real value')
23 print(y_test[0:10])
```

Listing .12: Kode model Decision Tree Regression.

5. Modelling — Decision Tree Regression

```

[15]
✓ Os
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import train_test_split

df_X = df.drop(['id', 'date', 'price'], axis=1)
df_y = df['price']

X = df_X.astype(float).values
y = df_y.astype(float).values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

dt = DecisionTreeRegressor(max_depth=10)
dt.fit(X_train, y_train)

print('coef of determination training', dt.score(X_train, y_train))
print('coef of determination testing', dt.score(X_test, y_test))

print('prediction')
y_pred = dt.predict(X_test)
print(y_pred[:10])
print('real value')
print(y_test[:10])

```

...

coef of determination training 0.9185274272472612
coef of determination testing 0.7742620332593732
prediction
[372512.5 837693.06862745 1087247.96153846 1984617.64705882
656314.08658009 237079.87179487 849106.21276596 517991.26363636
366270.875 519039.40166667]
real value
[365000. 865000. 1038000. 1490000. 711000. 211000. 790000. 680000.
384500. 605000.]

Gambar 16: Kode pemodelan Decision Tree Regression dengan max_depth=10.

Hasilnya R^2 training 0,9185 berbanding testing 0,7743: selisihnya cukup besar, yang menandakan pohon mulai overfitting meski kedalamannya sudah dibatasi.

```

5. Modelling — Decision Tree Regression

[15] ✓ Os ▶ from sklearn.tree import DecisionTreeRegressor
          from sklearn.model_selection import train_test_split

          df_X = df.drop(['id', 'date', 'price'], axis=1)
          df_y = df['price']

          X = df_X.astype(float).values
          y = df_y.astype(float).values

          X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

          dt = DecisionTreeRegressor(max_depth=10)
          dt.fit(X_train, y_train)

          print('coef of determination training', dt.score(X_train, y_train))
          print('coef of determination testing', dt.score(X_test, y_test))

          print('prediction')
          y_pred = dt.predict(X_test)
          print(y_pred[:10])
          print('real value')
          print(y_test[:10])

... coef of determination training 0.9185274272472612
     coef of determination testing 0.7742620332593732
prediction
[ 372512.5      837693.06862745 1087247.96153846 1984617.64705882
  656314.08658009 237079.87179487 849106.21276596 517991.26363636
  366270.875   519039.40166667]
real value
[ 365000.  865000. 1038000. 1490000.  711000.  211000.  790000.  680000.
 384500.  605000.]

```

Gambar 17: Output pelatihan dan prediksi Decision Tree Regression.

Evaluasi memakai kode yang identik dengan sebelumnya:

```

1 from sklearn.metrics import mean_squared_error
2 from sklearn.metrics import r2_score
3 import numpy as np
4
5 mse = mean_squared_error(y_test, y_pred)
6 rmse = np.sqrt(mse)
7 r2 = r2_score(y_test, y_pred)
8
9 print('rmse:', rmse)
10 print('r2:', r2)

```

Listing .13: Kode evaluasi Decision Tree.

```

  Evaluasi Decision Tree (RMSE & R2)

[16]
✓ 0s  from sklearn.metrics import mean_squared_error
      from sklearn.metrics import r2_score
      import numpy as np

      mse = mean_squared_error(y_test, y_pred)
      rmse = np.sqrt(mse)
      r2 = r2_score(y_test, y_pred)

      print('rmse:', rmse)
      print('r2:', r2)

  ... rmse: 180524.25684071498
      r2: 0.7742620332593732

```

Gambar 18: Kode evaluasi Decision Tree menggunakan RMSE dan R^2 .

Hasilnya RMSE 180.524,26 dan R^2 0,7743: lebih baik dari Linear Regression karena struktur pohon mampu menangkap pola non-linear pada data.

```

  Evaluasi Decision Tree (RMSE & R2)

[16]
✓ 0s  from sklearn.metrics import mean_squared_error
      from sklearn.metrics import r2_score
      import numpy as np

      mse = mean_squared_error(y_test, y_pred)
      rmse = np.sqrt(mse)
      r2 = r2_score(y_test, y_pred)

      print('rmse:', rmse)
      print('r2:', r2)

  ... rmse: 180524.25684071498
      r2: 0.7742620332593732

```

Gambar 19: Hasil evaluasi Decision Tree: RMSE dan R^2 lebih baik dibanding Linear Regression.

Visualisasi 50 sampel menunjukkan batang prediksi lebih rapat mengikuti nilai sebenarnya dibanding Linear Regression, menandakan error yang lebih kecil pada beberapa titik.

```

1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 data1 = pd.Series(y_test[:50].ravel())
5 data2 = pd.Series(y_pred[:50].ravel())
6
7 df_new = pd.concat([data1, data2], keys=['real values', 'predicted values'], axis=1)
8 df_new.plot(kind='bar', figsize=(15,3))
9

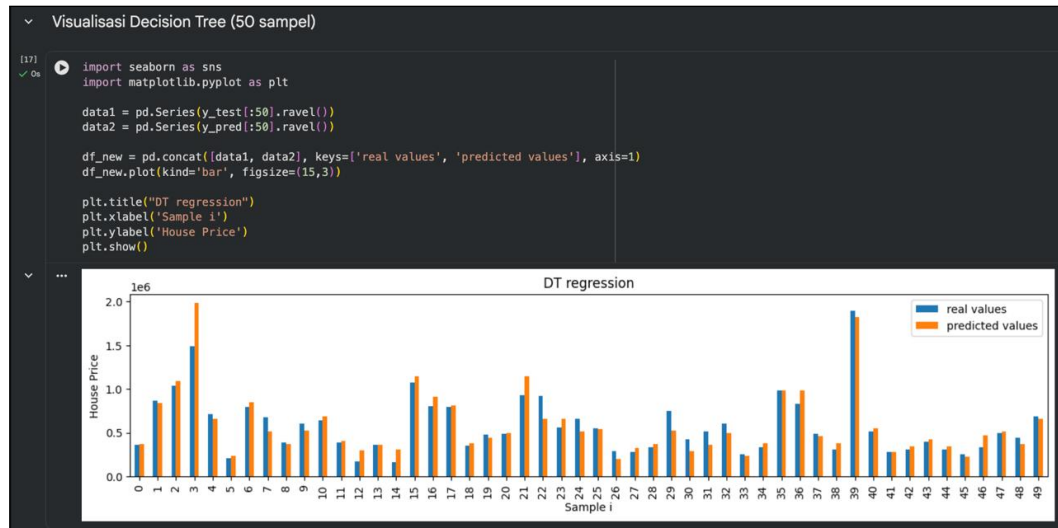
```

```

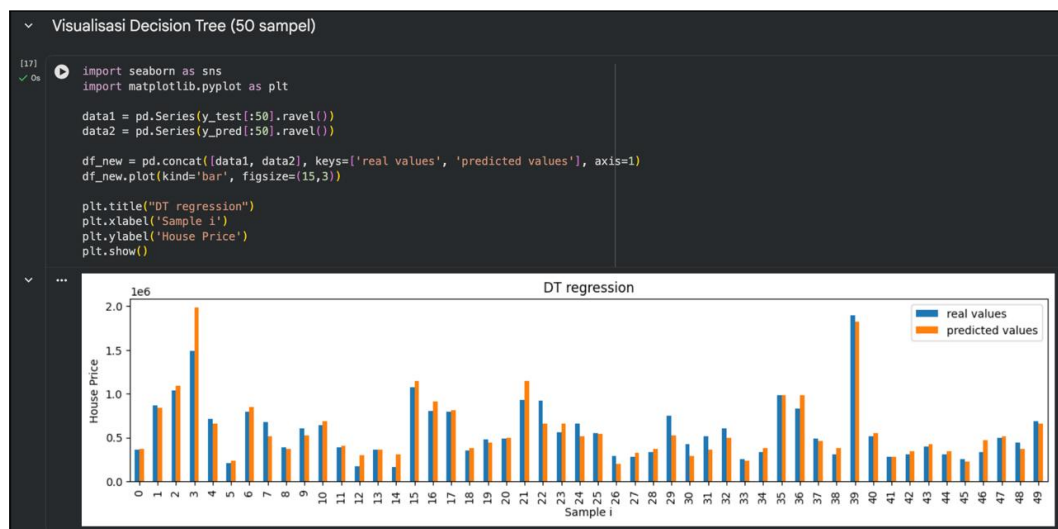
10 plt.title("DT regression")
11 plt.xlabel('Sample i')
12 plt.ylabel('House Price')
13 plt.show()

```

Listing .14: Kode visualisasi Decision Tree.



Gambar 20: Kode visualisasi perbandingan 50 nilai aktual dan prediksi Decision Tree.



Gambar 21: Grafik batang perbandingan real values dan predicted values untuk Decision Tree.

3.1.5.3 c. Random Forest Regression

Untuk Random Forest, langkahnya sama dengan inisialisasi `RandomForestRegressor()` memakai parameter default, yang secara default membangun sekumpulan pohon keputusan dan merata-ratakan prediksinya.

```

1 from sklearn.ensemble import RandomForestRegressor
2 from sklearn.model_selection import train_test_split
3
4 df_X = df.drop(['id', 'date', 'price'], axis=1)
5 df_y = df['price']
6
7 X = df_X.astype(float).values
8 y = df_y.astype(float).values
9
10 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
    ↪ random_state=42)
11
12 rf = RandomForestRegressor()
13 rf.fit(X_train, y_train)
14
15 print('coef of determination training', rf.score(X_train, y_train))
16 print('coef of determination testing', rf.score(X_test, y_test))
17
18 print('prediction')
19 y_pred = rf.predict(X_test)
20 print(y_pred[:10])
21 print('real value')
22 print(y_test[0:10])

```

Listing .15: Code model Random Forest Regression.

```
6. Modelling — Random Forest Regression

[18] ✓ 23s from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split

df_X = df.drop(['id', 'date', 'price'], axis=1)
df_y = df['price']

X = df_X.astype(float).values
y = df_y.astype(float).values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

rf = RandomForestRegressor()
rf.fit(X_train, y_train)

print('coef of determination training', rf.score(X_train, y_train))
print('coef of determination testing', rf.score(X_test, y_test))

print('prediction')
y_pred = rf.predict(X_test)
print(y_pred[:10])
print('real value')
print(y_test[:10])

... coef of determination training 0.9819848595528915
coef of determination testing 0.8556886509231035
prediction
[ 378807.    905734.26  1125072.8  2104427.    709517.32  245677.06
   841769.5   616474.28  405608.13  547626.73]
real value
[ 365000.   865000.  1038000.  1490000.   711000.   211000.   790000.   680000.
   384500.   605000.]
```

Gambar 22: Kode pemodelan Random Forest Regression dengan parameter default.

Hasilnya R^2 training 0,9820 berbanding testing 0,8557. Nilai training yang tinggi wajar untuk ensemble; yang lebih penting, skor testingnya tetap tertinggi di antara ketiga model sehingga generalisasinya paling baik.

6. Modelling — Random Forest Regression

```

[18] ✓ 23s from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split

df_X = df.drop(['id', 'date', 'price'], axis=1)
df_y = df['price']

X = df_X.astype(float).values
y = df_y.astype(float).values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

rf = RandomForestRegressor()
rf.fit(X_train, y_train)

print('coef of determination training', rf.score(X_train, y_train))
print('coef of determination testing', rf.score(X_test, y_test))

print('prediction')
y_pred = rf.predict(X_test)
print(y_pred[:10])
print('real value')
print(y_test[:10])

... coef of determination training 0.9819848595528915
coef of determination testing 0.8556886509231035
prediction
[ 378807.    905734.26  1125072.8   2104427.    709517.32  245677.06
   841769.5   616474.28  405608.13  547626.73]
real value
[ 365000.   865000.  1038000.  1490000.   711000.   211000.   790000.   680000.
   384500.   605000.]

```

Gambar 23: Output pelatihan dan prediksi Random Forest Regression.

```

1 from sklearn.metrics import mean_squared_error
2 from sklearn.metrics import r2_score
3 import numpy as np
4
5 mse = mean_squared_error(y_test, y_pred)
6 rmse = np.sqrt(mse)
7 r2 = r2_score(y_test, y_pred)
8
9 print('rmse:', rmse)
10 print('r2:', r2)

```

Listing .16: Kode evaluasi Random Forest.

```

  Evaluasi Random Forest (RMSE & R2)

[19]
✓ 0s  from sklearn.metrics import mean_squared_error
      from sklearn.metrics import r2_score
      import numpy as np

      mse = mean_squared_error(y_test, y_pred)
      rmse = np.sqrt(mse)
      r2 = r2_score(y_test, y_pred)

      print('rmse:', rmse)
      print('r2:', r2)

  ... rmse: 144338.9376993634
      r2: 0.8556886509231035

```

Gambar 24: Kode evaluasi Random Forest menggunakan RMSE dan R^2 .

Hasilnya RMSE 144.338,94 dan R^2 0,8557: RMSE terendah dan R^2 tertinggi di antara ketiga model, membuktikan keunggulan metode ensemble dalam menangkap pola data yang kompleks.

```

  Evaluasi Random Forest (RMSE & R2)

[19]
✓ 0s  from sklearn.metrics import mean_squared_error
      from sklearn.metrics import r2_score
      import numpy as np

      mse = mean_squared_error(y_test, y_pred)
      rmse = np.sqrt(mse)
      r2 = r2_score(y_test, y_pred)

      print('rmse:', rmse)
      print('r2:', r2)

  ... rmse: 144338.9376993634
      r2: 0.8556886509231035

```

Gambar 25: Hasil evaluasi Random Forest: RMSE terendah dan R^2 tertinggi.

```

1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 data1 = pd.Series(y_test[:50].ravel())
5 data2 = pd.Series(y_pred[:50].ravel())
6
7 df_new = pd.concat([data1, data2], keys=['real values', 'predicted values'], axis=1)
8 df_new.plot(kind='bar', figsize=(15,3))
9
10 plt.title("RF regression")

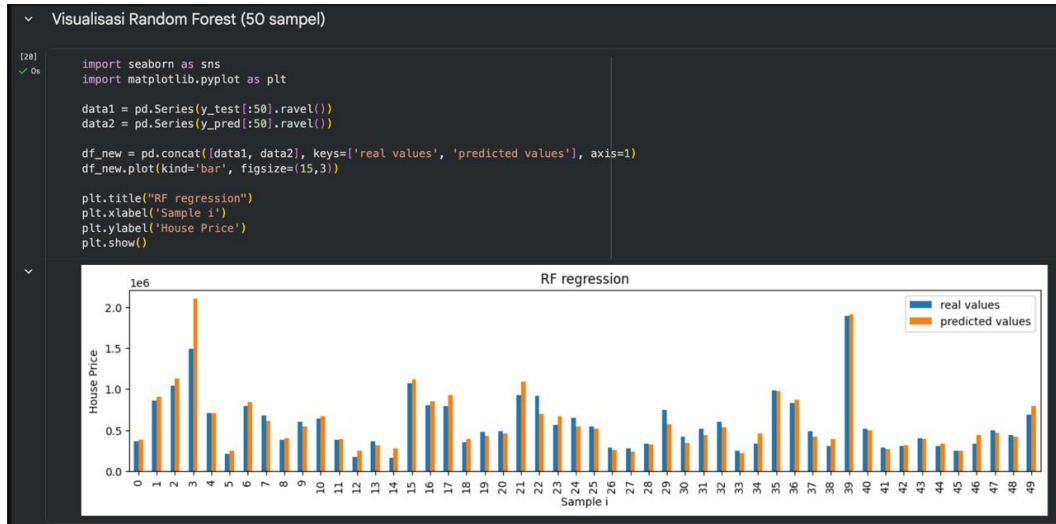
```

```

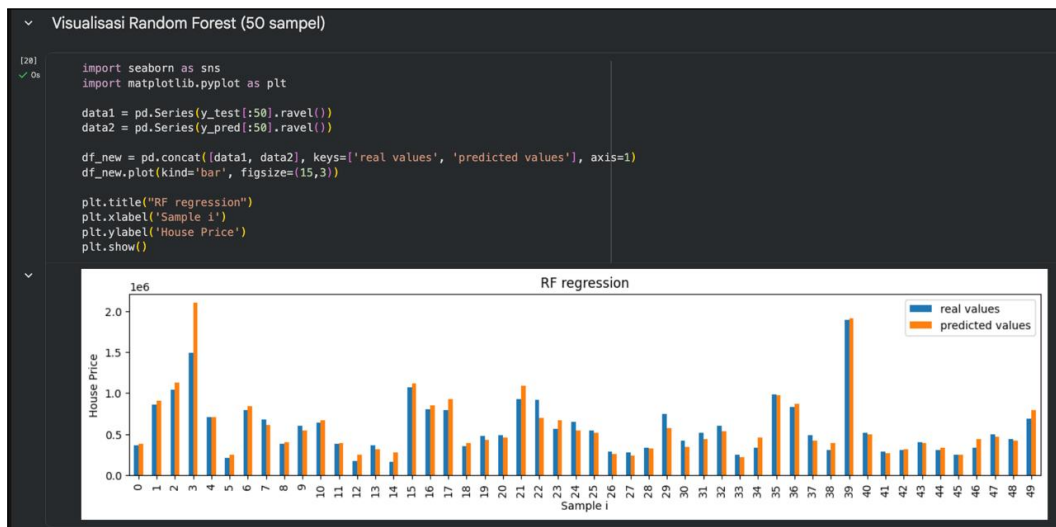
11 plt.xlabel('Sample i')
12 plt.ylabel('House Price')
13 plt.show()

```

Listing .17: Kode visualisasi Random Forest.



Gambar 26: Kode visualisasi perbandingan 50 nilai aktual dan prediksi Random Forest.



Gambar 27: Grafik batang perbandingan real values dan predicted values untuk Random Forest: batang prediksi hampir berimpit dengan nilai sebenarnya.

3.1.6 Perbandingan Performa Model KC House

Ringkasan hasil eksperimen ketiga model pada dataset KC House (verifikasi lokal Python 3.11, scikit-learn 1.9.0, split 70:30) saya susun pada Tabel .1.

Model	RMSE	R2
Linear Regression	208296.73	0.6995
Decision Tree (max_depth=10)	180524.26	0.7743
Random Forest	144338.94	0.8557

Table .1: Perbandingan performa tiga model regresi pada dataset KC House (output Google Colab).

Urutannya konsisten: Random Forest terbaik di semua metrik, Decision Tree di tengah, dan Linear Regression paling rendah. Wajar saja, karena hubungan harga rumah terhadap fiturnya banyak yang non-linear (interaksi luas, grade, lokasi), sehingga model pohon yang fleksibel mengungguli garis linier. Selisih R^2 training dan testing juga menarik untuk dicermati: Linear Regression hampir tidak overfitting (0,6995 vs 0,6995), sedangkan Decision Tree (0,9185 vs 0,7743) dan Random Forest (0,9820 vs 0,8557) menunjukkan gap yang wajar untuk model berkapasitas besar.

3.1.7 Tugas dan Analisis: Car Price Prediction

Sesuai Modul 4.5, saya mengerjakan tugas tambahan menggunakan dataset *Car Price Prediction* (<https://www.kaggle.com/datasets/hellbuoy/car-price-prediction/data>).

3.1.7.1 Tujuan Penggunaan Dataset

Dataset Car Price Prediction digunakan untuk memprediksi harga mobil berdasarkan spesifikasi teknisnya, agar perusahaan konsultan otomotif dapat menetapkan harga jual yang kompetitif tanpa harus melakukan survei manual untuk setiap unit. Dataset klasik ini (205 baris, 26 kolom) juga sering dipakai sebagai studi kasus regresi linier berganda karena menggabungkan fitur numerik (dimensi, mesin, konsumsi BBM) dan kategorikal (jenis bahan bakar, bodi, penggerak) [13, 14].

3.1.7.2 Atribut Input dan Output

Berdasarkan eksplorasi awal dataset, atribut dapat didefinisikan sebagai berikut:

- **Atribut input (fitur):** seluruh kolom kecuali price — dimensi (wheelbase, carlength, carwidth, carheight, curbweight), mesin (enginesize, boreratio, stroke, compressionratio, horsepower, peakrpm), konsumsi BBM (citympg, highwaympg), simbol risiko (symboling), serta kolom kategorikal (fueltype, aspiration, doornumber, carbody, drivewheel, enginelocation, enginetype, cylindernumber, fuelsystem) yang saya encoding. Kolom car_ID dan CarName saya buang sebagai identifier/nama unit yang tidak general.

- **Atribut output (target):** price (harga mobil, kontinu; rata-rata 13.276,71, rentang 5.118–45.400).

Atribut	Penjelasan
car_ID	Identifier unik unit mobil (dibuang saat pem...
symboling	Tingkat risiko asuransi mobil.
CarName	Nama/merek unit mobil (dibuang, terlalu spe...
fueltype	Jenis bahan bakar (gas/diesel).
aspiration	Tipe aspirasi mesin (std/turbo).
doornumber	Jumlah pintu (two/four).
carbody	Bentuk bodi (sedan/hatchback/wagon/dsb).
drivewheel	Penggerak roda (fwd/rwd/4wd).
enginelocation	Posisi mesin (front/rear).
wheelbase, carlength, carwidth, carheight, curbweight	Dimensi dan bobot mobil.
enginetype, cylindernumber, enginesize, fuelsystem	Spesifikasi mesin dan sistem bahan bakar.
boreratio, stroke, compressionratio, horsepower, peakrpm	Parameter performa mesin.
citympg, highwaympg	Konsumsi BBM dalam kota dan jalan tol.
price	Target: harga mobil (kontinu).

Table .2: Deskripsi atribut dataset Car Price Prediction.

3.1.7.3 Eksplorasi Awal dan Preprocessing Dataset Car

Saya memuat dataset dari mirror GitHub dan melakukan EDA singkat: memeriksa head, info, deskripsi statistik, dan korelasi. Tidak ada missing value di seluruh kolom. Korelasi tertinggi dengan price adalah enginesize (0,87), curbweight (0,84), dan horsepower (0,81) — spesifikasi mesin dan bobot paling menentukan harga mobil.

```

1 car_url = 'https://raw.githubusercontent.com/SampattKumar/Linear-Regression-Car-
  ↳ Dataset/master/CarPrice_Assignment.csv' # mirror GitHub dari Kaggle
2 car_df = pd.read_csv(car_url)
3 print(car_df.shape) # (205, 26)
4 print(car_df.head())
5 print(car_df.info())
6 print(car_df.describe())
7 print(car_df.isnull().sum()) # 0 missing di semua kolom
8 print(car_df.corr(numeric_only=True) ['price'] .drop('price') .sort_values(ascending=
  ↳ False) .head(6))

```

Listing .18: Memuat dan memeriksa dataset Car.

```

7a. Load dan EDA Dataset Car

car_url = 'https://raw.githubusercontent.com/SampattKumar/Linear-Regression-Car-Dataset/master/CarPrice_Assignment.csv'
car_df = pd.read_csv(car_url)
print(car_df.shape)
print(car_df.head())
print(car_df.info())
print(car_df.describe())
print(car_df.isnull().sum())
print(car_df.corr(numeric_only=True)['price'].drop('price').sort_values(ascending=False).head(6))

(205, 26)
car_ID  symboling      CarName  fueltype  aspiration  doornumber  \
0      1          3      alfa-romero  giulia      gas         std         two  \
1      2          3      alfa-romero  stelvio     gas         std         two  \
2      3          1  alfa-romero  Quadrifoglio  gas         std         two  \
3      4          2      audi 100  ls         gas         std         four  \
4      5          2      audi 100ls         gas         std         four

carbody  drivewheel  enginelocation  wheelbase  ...  enginesize  \
0  convertible     rwd           front      88.6  ...      130  \
1  convertible     rwd           front      88.6  ...      130  \
2  hatchback      rwd           front      94.5  ...      152  \
3  sedan          fwd           front      99.8  ...      109  \
4  sedan          4wd           front      99.4  ...      136

fuelsystem  boreratio  stroke  compressionratio  horsepower  peakrpm  citympg  \
0  mpfi          3.47    2.68           9.0         111      5000    21  \
1  mpfi          3.47    2.68           9.0         111      5000    21  \
2  mpfi          2.68    3.47           9.0         154      5000    19  \
3  mpfi          3.19    3.40          10.0         102      5500    24  \
4  mpfi          3.19    3.40           8.0         115      5500    18

highwaympg  price
0          27  13495.0
1          27  16500.0
2          26  16500.0
3          30  13950.0
4          22  17450.0

```

Gambar 28: Lima baris pertama dataset Car Price beserta info kolom.

None						
	car_ID	symboling	wheelbase	carlength	carwidth	carheight
count	205.000000	205.000000	205.000000	205.000000	205.000000	205.000000
mean	103.000000	0.834146	98.756585	174.049268	65.907805	53.724878
std	59.322565	1.245307	6.021776	12.337289	2.145204	2.443522
min	1.000000	-2.000000	86.600000	141.100000	60.300000	47.800000
25%	52.000000	0.000000	94.500000	166.300000	64.100000	52.000000
50%	103.000000	1.000000	97.000000	173.200000	65.500000	54.100000
75%	154.000000	2.000000	102.400000	183.100000	66.900000	55.500000
max	205.000000	3.000000	120.900000	208.100000	72.300000	59.800000
	curbweight	enginesize	boreratio	stroke	compressionratio	\
count	205.000000	205.000000	205.000000	205.000000	205.000000	
mean	2555.565854	126.907317	3.329756	3.255415	10.142537	
std	520.680204	41.642693	0.270844	0.313597	3.972040	
min	1488.000000	61.000000	2.540000	2.070000	7.000000	
25%	2145.000000	97.000000	3.150000	3.110000	8.600000	
50%	2414.000000	120.000000	3.310000	3.290000	9.000000	
75%	2935.000000	141.000000	3.580000	3.410000	9.400000	
max	4066.000000	326.000000	3.940000	4.170000	23.000000	
	horsepower	peakrpm	citympg	highwaympg	price	
count	205.000000	205.000000	205.000000	205.000000	205.000000	
mean	104.117073	5125.121951	25.219512	30.751220	13276.710571	
std	39.544167	476.985643	6.542142	6.886443	7988.852332	
min	48.000000	4150.000000	13.000000	16.000000	5118.000000	
25%	70.000000	4800.000000	19.000000	25.000000	7788.000000	
50%	95.000000	5200.000000	24.000000	30.000000	10295.000000	
75%	116.000000	5500.000000	30.000000	34.000000	16503.000000	
max	288.000000	6600.000000	49.000000	54.000000	45400.000000	

car_ID	0
symboling	0
CarName	0
fueltype	0
aspiration	0
doornumber	0
carbody	0
drivewheel	0
enginelocation	0
wheelbase	0
carlength	0
carwidth	0
carheight	0
curbweight	0
enginetype	0
cylindernumber	0
enginesize	0
fuelsystem	0
boreratio	0
stroke	0
compressionratio	0
horsepower	0
peakrpm	0
citympg	0
highwaympg	0
price	0
dtype: int64	
enginesize	0.874145
curbweight	0.835305
horsepower	0.808139
carwidth	0.759325
carlength	0.682920
wheelbase	0.577816
Name: price, dtype: float64	

Gambar 29: Hasil EDA dataset Car: deskripsi statistik dan korelasi tertinggi terhadap price (enginesize, curbweight, horsepower).

Untuk preprocessing, saya melakukan: (1) membuang car_ID dan CarName, (2) *one-hot encoding* kolom kategorikal dengan `pd.get_dummies(drop_first=True)` yang meng-

hasilkan 43 fitur numerik, (3) split 70:30, dan (4) StandardScaler yang di-fit hanya pada data train — konsisten dengan workflow P3 yang memakai `.astype(int)` setelah `get_dummies` agar kolom boolean tidak menghasilkan NaN saat scaling.

```

1 from sklearn.preprocessing import StandardScaler
2
3 car2 = car_df.drop(['car_ID', 'CarName'], axis=1)
4 X_car = pd.get_dummies(car2.drop(['price'], axis=1), drop_first=True).astype(int)
5 y_car = car2['price'].astype(float).values # hasil: (205, 43) fitur numerik
6 print('X_car shape:', X_car.shape)
7
8 X_train_c, X_test_c, y_train_c, y_test_c = train_test_split(X_car.values, y_car,
9 ↪ test_size=0.3, random_state=42)
10 scaler_c = StandardScaler().fit(X_train_c)
11 X_train_cs = scaler_c.transform(X_train_c)
12 X_test_cs = scaler_c.transform(X_test_c)
13 print('mean train ~0:', X_train_cs.mean().round(6))

```

Listing .19: Preprocessing dataset Car.

```

7b. Preprocessing Dataset Car

from sklearn.preprocessing import StandardScaler

car2 = car_df.drop(['car_ID', 'CarName'], axis=1)
X_car = pd.get_dummies(car2.drop(['price'], axis=1), drop_first=True).astype(int)
y_car = car2['price'].astype(float).values
print('X_car shape:', X_car.shape)

X_train_c, X_test_c, y_train_c, y_test_c = train_test_split(X_car.values, y_car, test_size=0.3, random_state=42)
scaler_c = StandardScaler().fit(X_train_c)
X_train_cs = scaler_c.transform(X_train_c)
X_test_cs = scaler_c.transform(X_test_c)
print('mean train ~0:', X_train_cs.mean().round(6))

... X_car shape: (205, 43)
    mean train ~0: 0.0

```

Gambar 30: Hasil preprocessing dataset Car: bentuk fitur (205, 43) dan hasil scaling dengan mean mendekati nol.

3.1.7.4 Pemodelan Regresi pada Dataset Car

Linear Regression:

```

1 model = LinearRegression()
2 model.fit(X_train_cs, y_train_c)
3 y_pred_c = model.predict(X_test_cs)
4 rmse_c = np.sqrt(mean_squared_error(y_test_c, y_pred_c))
5 r2_c = r2_score(y_test_c, y_pred_c)
6 r_c = np.corrcoef(y_test_c, y_pred_c)[0, 1]
7 print('train R2:', model.score(X_train_cs, y_train_c))
8 print('test R2:', model.score(X_test_cs, y_test_c))
9 print('rmse:', rmse_c)
10 print('r2:', r2_c)
11 print('r:', r_c)

```

Listing .20: Linear Regression pada dataset Car.

7c. Modelling Car — Linear Regression

```
model = LinearRegression()
model.fit(X_train_cs, y_train_c)
y_pred_c = model.predict(X_test_cs)
rmse_c = np.sqrt(mean_squared_error(y_test_c, y_pred_c))
r2_c = r2_score(y_test_c, y_pred_c)
r_c = np.corrcoef(y_test_c, y_pred_c)[0, 1]
print('train R2:', model.score(X_train_cs, y_train_c))
print('test R2:', model.score(X_test_cs, y_test_c))
print('rmse:', rmse_c)
print('r2:', r2_c)
print('r:', r_c)

train R2: 0.9458004903831513
test R2: 0.8823874081671385
rmse: 2854.595477821965
r2: 0.8823874081671385
r: 0.9400361191342694
```

Gambar 31: Performa Linear Regression pada dataset Car (RMSE 2854,60, R2 0,8824).

Decision Tree (max_depth=10):

```
1 model = DecisionTreeRegressor(max_depth=10, random_state=42)
2 model.fit(X_train_cs, y_train_c)
3 y_pred_c = model.predict(X_test_cs)
4 rmse_c = np.sqrt(mean_squared_error(y_test_c, y_pred_c))
5 r2_c = r2_score(y_test_c, y_pred_c)
6 r_c = np.corrcoef(y_test_c, y_pred_c)[0, 1]
7 print('train R2:', model.score(X_train_cs, y_train_c))
8 print('test R2:', model.score(X_test_cs, y_test_c))
9 print('rmse:', rmse_c)
10 print('r2:', r2_c)
11 print('r:', r_c)
```

Listing .21: Decision Tree pada dataset Car.

```
7d. Modelling Car — Decision Tree

[24]
✓ Os ▶ model = DecisionTreeRegressor(max_depth=10, random_state=42)
      model.fit(X_train_cs, y_train_c)
      y_pred_c = model.predict(X_test_cs)
      rmse_c = np.sqrt(mean_squared_error(y_test_c, y_pred_c))
      r2_c = r2_score(y_test_c, y_pred_c)
      r_c = np.corrcoef(y_test_c, y_pred_c)[0, 1]
      print('train R2:', model.score(X_train_cs, y_train_c))
      print('test R2:', model.score(X_test_cs, y_test_c))
      print('rmse:', rmse_c)
      print('r2:', r2_c)
      print('r:', r_c)

... train R2: 0.9983808255200233
    test R2: 0.8850031039843462
    rmse: 2822.6739951100008
    r2: 0.8850031039843462
    r: 0.9510126217644745
```

Gambar 32: Performa Decision Tree pada dataset Car (RMSE 2822,67, R2 0,8850).

Random Forest:

```
1 model = RandomForestRegressor(random_state=42)
2 model.fit(X_train_cs, y_train_c)
3 y_pred_c = model.predict(X_test_cs)
4 rmse_c = np.sqrt(mean_squared_error(y_test_c, y_pred_c))
5 r2_c = r2_score(y_test_c, y_pred_c)
6 r_c = np.corrcoef(y_test_c, y_pred_c)[0, 1]
7 print('train R2:', model.score(X_train_cs, y_train_c))
8 print('test R2:', model.score(X_test_cs, y_test_c))
9 print('rmse:', rmse_c)
10 print('r2:', r2_c)
11 print('r:', r_c)
```

Listing .22: Random Forest pada dataset Car.

[25]
✓ Os



Gambar 33: Performa Random Forest pada dataset Car (RMSE 1984,06, R2 0,9432).

3.1.7.5 Tabel Performa Model

```

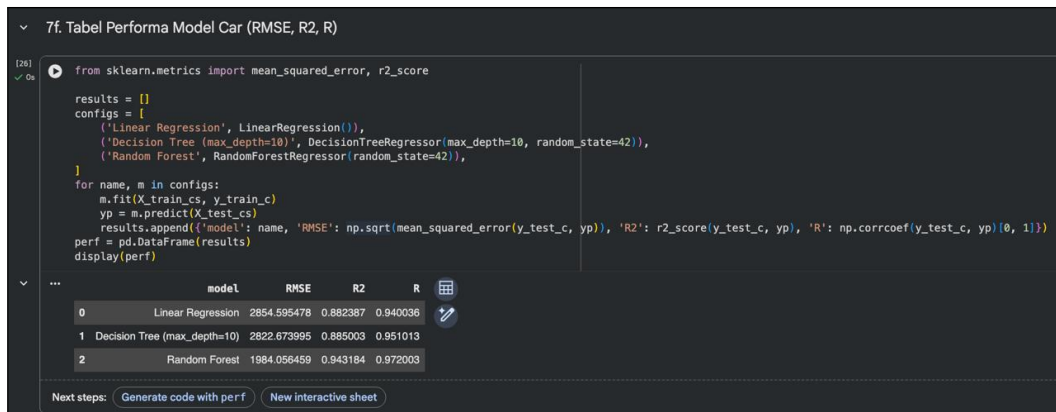
1 from sklearn.metrics import mean_squared_error, r2_score
2
3 results = []
4 configs = [
5     ('Linear Regression', LinearRegression()),
6     ('Decision Tree (max_depth=10)', DecisionTreeRegressor(max_depth=10,
7     ↪ random_state=42)),
8     ('Random Forest', RandomForestRegressor(random_state=42)),
9 ]
10 for name, m in configs:
11     m.fit(X_train_cs, y_train_c)
12     yp = m.predict(X_test_cs)
13     results.append({'model': name, 'RMSE': np.sqrt(mean_squared_error(y_test_c, yp))
14     ↪ , 'R2': r2_score(y_test_c, yp), 'R': np.corrcoef(y_test_c, yp)[0, 1]})
15 perf = pd.DataFrame(results)
16 display(perf)

```

Listing .23: Tabel performa model Car.

Model	RMSE	R2	R
Linear Regression	2854.60	0.8824	0.9400
Decision Tree (max_depth=10)	2822.67	0.8850	0.9510
Random Forest	1984.06	0.9432	0.9720

Table .3: Perbandingan performa tiga model regresi pada dataset Car Price Prediction (hasil verifikasi lokal Python 3.11, scikit-learn 1.9.0, split 70:30, StandardScaler).



The screenshot shows a Jupyter Notebook cell with the following Python code:

```
[28] from sklearn.metrics import mean_squared_error, r2_score

results = []
configs = [
    ('Linear Regression', LinearRegression()),
    ('Decision Tree (max_depth=10)', DecisionTreeRegressor(max_depth=10, random_state=42)),
    ('Random Forest', RandomForestRegressor(random_state=42)),
]

for name, m in configs:
    m.fit(X_train_c, y_train_c)
    yp = m.predict(X_test_c)
    results.append({'model': name, 'RMSE': np.sqrt(mean_squared_error(y_test_c, yp)), 'R2': r2_score(y_test_c, yp), 'R': np.corrcoef(y_test_c, yp)[0, 1]})

perf = pd.DataFrame(results)
display(perf)
```

The output is a DataFrame with the following data:

	model	RMSE	R2	R
0	Linear Regression	2854.595478	0.882387	0.940036
1	Decision Tree (max_depth=10)	2822.673995	0.885003	0.951013
2	Random Forest	1984.056459	0.943184	0.972003

Gambar 34: Tabel performa model yang dihasilkan dari kode Python (output DataFrame).

3.1.7.6 Analisis Model Terbaik

Berdasarkan eksperimen pada dataset Car (Tabel .3), model **Random Forest** menunjukkan performa terbaik secara keseluruhan dengan RMSE terendah 1.984,06, R2 tertinggi 0,9432, dan korelasi R tertinggi 0,9720. Keunggulan ini konsisten dengan karakteristik ensemble yang merata-ratakan banyak pohon sehingga meredam variansi pohon tunggal dan menangkap interaksi non-linear antar spesifikasi mobil. Decision Tree berada di posisi kedua (RMSE 2.822,67, R2 0,8850) dan Linear Regression ketiga (RMSE 2.854,60, R2 0,8824); selisih keduanya tipis karena hubungan harga mobil terhadap spesifikasi seperti enginesize dan curbweight memang cenderung linear, sehingga model sederhana sudah kompetitif, namun ensemble tetap menang dengan margin yang jelas.

Pola yang sama terlihat pada percobaan KC House (Tabel .1): Random Forest terbaik (RMSE 144.338,94, R2 0,8557), diikuti Decision Tree (RMSE 180.524,26, R2 0,7743), dan Linear Regression terakhir (RMSE 208.296,73, R2 0,6995). Jadi pada kedua dataset, Random Forest menjadi pilihan terbaik dengan alasan yang sama: ensemble mampu menangkap pola kompleks tanpa mengorbankan generalisasi.

Kesimpulan

Berdasarkan praktikum yang telah saya lakukan pada Pertemuan 4, saya memahami beberapa hal berikut:

1. Regresi adalah teknik supervised learning untuk memprediksi target numerik yang kontinu. Saya telah memahami konsep dasarnya: model mempelajari fungsi matematis dari pasangan input-output berlabel untuk memprediksi nilai baru, berbeda dengan klasifikasi yang keluarannya label diskrit dan clustering yang tidak memakai label.
2. Saya telah mempelajari prinsip kerja tiga algoritma regresi: Linear Regression (membangun garis/hyperplane terbaik dengan Least Squares yang meminimalkan SSE, cepat dan interpretable tetapi terbatas pada pola linear dan sensitif terhadap outlier), Decision Tree Regression (membagi data secara rekursif berdasarkan split yang meminimalkan MSE, efektif untuk pola non-linear tetapi rawan overfitting sehingga perlu `max_depth` atau pruning), dan Random Forest Regression (ensemble bootstrap yang merata-ratakan banyak pohon sehingga stabil, akurat, dan memberi feature importance dengan biaya komputasi lebih besar).
3. Saya berhasil menerapkan ketiga algoritma tersebut pada dataset KC House (21.613 baris, 21 kolom) dengan workflow yang benar: EDA (deskripsi statistik, histogram, korelasi, cek missing value, cek kategorikal), pemisahan fitur/target dengan membuang `id` dan `date`, konversi float, dan split 70:30 dengan `random_state=42`.
4. Evaluasi model dengan RMSE, R^2 , dan korelasi R menunjukkan hasil yang konsisten pada kedua dataset: pada KC House, Random Forest terbaik (RMSE 144.338,94, R^2 0,8557), diikuti Decision Tree (RMSE 180.524,26, R^2 0,7743), dan Linear Regression (RMSE 208.296,73, R^2 0,6995); pada tugas Car Price Prediction, Random Forest juga terbaik (RMSE 1.984,06, R^2 0,9432, R 0,9720). Saya juga memahami mengapa accuracy/precision/recall tidak dipakai untuk regresi: metrik tersebut membutuhkan konsep benar/salah dari target diskrit, sedangkan prediksi regresi bersifat kontinu sehingga kualitasnya harus diukur dari besarnya error (RMSE) dan proporsi variansi yang dijelaskan (R^2).

Praktikum ini memberi saya dasar untuk memilih, melatih, dan mengevaluasi model regresi

secara objektif, dengan mempertimbangkan RMSE, R^2 , dan korelasi secara bersamaan alih-alih berhenti pada satu metrik.

Daftar Pustaka

- [1] J. Han, J. Pei, and H. Tong, *Data Mining: Concepts and Techniques*, 4th ed. Morgan Kaufmann/Elsevier, 2022.
- [2] Python Software Foundation, “Python 3 Documentation.” [Online]. Tersedia: <https://www.python.org/doc/>.
- [3] The pandas Development Team, “pandas.DataFrame.” [Online]. Tersedia: <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html>.
- [4] NumPy Developers, “NumPy Documentation.” [Online]. Tersedia: <https://numpy.org/>.
- [5] Scikit-learn Developers, “Linear Models.” [Online]. Tersedia: https://scikit-learn.org/stable/modules/linear_model.html.
- [6] Scikit-learn Developers, “Decision Trees.” [Online]. Tersedia: <https://scikit-learn.org/stable/modules/tree.html>.
- [7] Scikit-learn Developers, “Ensemble methods.” [Online]. Tersedia: <https://scikit-learn.org/stable/modules/ensemble.html>.
- [8] Scikit-learn Developers, “Metrics and scoring: quantifying the quality of predictions.” [Online]. Tersedia: https://scikit-learn.org/stable/modules/model_evaluation.html.
- [9] Scikit-learn Developers, “Preprocessing data.” [Online]. Tersedia: <https://scikit-learn.org/stable/modules/preprocessing.html>.
- [10] IBM, “Apa Itu Analisis Data Eksplorasi?” [Online]. Tersedia: <https://www.ibm.com/id-id/think/topics/exploratory-data-analysis>.
- [11] Amazon Web Services, “Apa itu Regresi Linier?” [Online]. Tersedia: <https://aws.amazon.com/id/what-is/linear-regression/>.
- [12] S. Chandel, “KC House Data.” Kaggle. [Online]. Tersedia: <https://www.kaggle.com/datasets/shivachandel/kc-house-data>.

- [13] Hellbuoy, "Car Price Prediction." Kaggle. [Online]. Tersedia: <https://www.kaggle.com/datasets/hellbuoy/car-price-prediction/data>.
- [14] S. Kumar, "Linear Regression Car Dataset (CarPrice_Assignment.csv)." GitHub. [Online]. Tersedia: https://github.com/SampattKumar/Linear-Regression-Car-Dataset/blob/master/CarPrice_Assignment.csv.
- [15] Dicoding Indonesia, "Rangkuman Supervised Learning - Regresi." [Online]. Tersedia: <https://www.dicoding.com/academies/184/tutorials/38833>.
- [16] Dicoding Indonesia, "Machine Learning Workflow: Di Balik Model AI yang Sukses," 23 Oktober 2024. [Online]. Tersedia: <https://www.dicoding.com/blog/machine-learning-workflow-langkah-langkah-praktis-untuk-membangun-model-ai/>.
- [17] Trivusi, "Apa Bedanya Artificial Intelligence, Machine Learning, dan Deep Learning?" 13 Maret 2022. [Online]. Tersedia: <https://www.trivusi.web.id/2022/03/perbedaan-ai-ml-dl.html>.
- [18] ITBOX, "Algoritma Machine Learning: Pengertian, Penerapan, dan Contohnya," 23 Februari 2024. [Online]. Tersedia: <https://itbox.id/blog/algoritma-machine-learning-pengertian-penerapan-dan-contohnya/>.
- [19] Digital Skola, "Data Preprocessing: Definisi, Tahapan, dan Implementasinya." [Online]. Tersedia: <https://digitalskola.com/blog/data-science/data-preprocessing>.